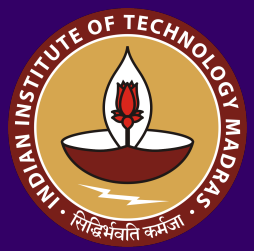
Scientific Computing & Advanced Programming Lab

SAURAV SAMANTARAY

Department of Mathematics
IIT Madras

Lecture-1



sauravsray@iitm.ac.in

Welcome!

- Slides will be available at ”https://sauravsray.github.io/MA-5105-Aug-26/”
- Codes corresponding to each lecture will be available at the same place.

Prerequisites and Demands

Prerequisites

- Some python programming experience
- Basic linear algebra and calculus

Prerequisites and Demands

Prerequisites

- Some python programming experience
- Basic linear algebra and calculus

Demands

- honesty: don't cheat yourself !

Prerequisites and Demands

Prerequisites

- Some python programming experience
- Basic linear algebra and calculus

Demands

- honesty: don't cheat yourself !
- clocking numerous hours per week in writing / testing codes.

Prerequisites and Demands

Prerequisites

- Some python programming experience
- Basic linear algebra and calculus

Demands

- honesty: don't cheat yourself !
- clocking numerous hours per week in writing / testing codes.
- interact with me and the teaching assistant

Course Policies

Collaboration

- struggle first

Course Policies

Collaboration

- struggle first
- discuss specific doubts over the macroscopic problem

Course Policies

Collaboration

- struggle first
- discuss specific doubts over the macroscopic problem
- don't copy anyone's code

Course Policies

Collaboration

- struggle first
- discuss specific doubts over the macroscopic problem
- don't copy anyone's code

Course Policies

Collaboration

- struggle first
- discuss specific doubts over the macroscopic problem
- don't copy anyone's code

LLMS Usage

- don't use to solve the problem

Course Policies

Collaboration

- struggle first
- discuss specific doubts over the macroscopic problem
- don't copy anyone's code

LLMS Usage

- don't use to solve the problem
- may be only ask for clarifications

Course Policies

Collaboration

- struggle first
- discuss specific doubts over the macroscopic problem
- don't copy anyone's code

LLMS Usage

- don't use to solve the problem
- may be only ask for clarifications

Assignments

- No assignments

Course Policies

Collaboration

- struggle first
- discuss specific doubts over the macroscopic problem
- don't copy anyone's code

LLMS Usage

- don't use to solve the problem
- may be only ask for clarifications

Assignments

- No assignments
- A lot of problem sets

Grading

- 4 surprise quizzes (fairly easy), 10 Marks each.
- 1 viva exam, 20 Marks.
- Class participation: 20 Marks.
- A final project worth 20 Marks.

Computing

- What do computers do ?

Computing

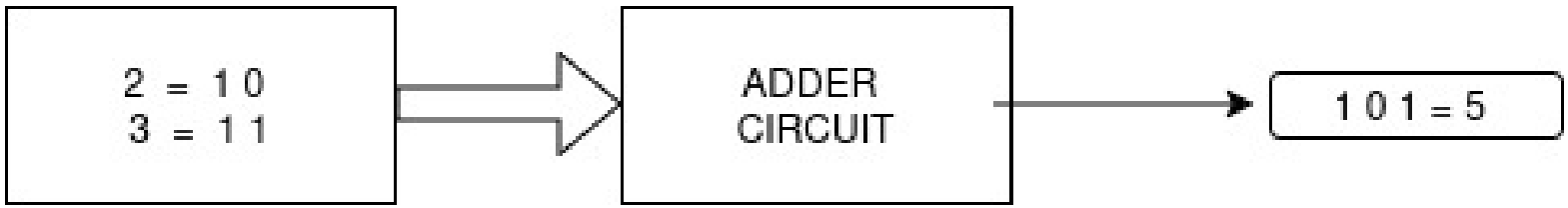
- What do computers do ?
- How do they compute ?

Computing

- What do computers do ?
- How do they compute ?
  1. Data movement: read / store
  2. logical : Compare / And / Or
  3. add / subtract / Multiply / Divide / Increment / Decrement
  4. control flow: Jump to another instruction

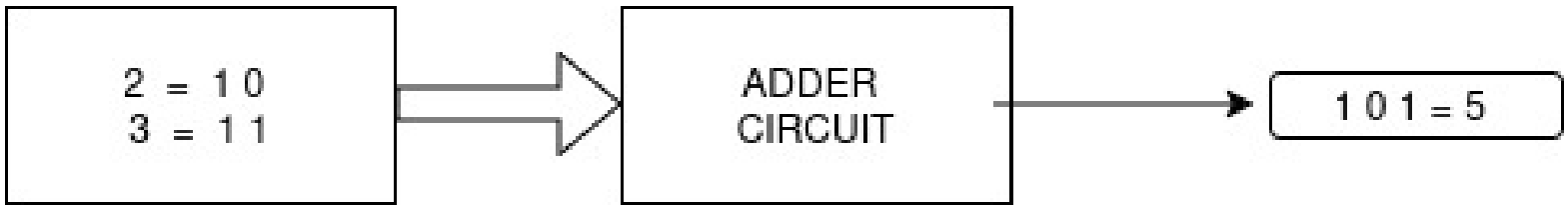
Computing

- What do computers do ?
- How do they compute ?
  1. Data movement: read / store
  2. logical : Compare / And / Or
  3. add / subtract / Multiply / Divide / Increment / Decrement
  4. control flow: Jump to another instruction
- Example-1: Calculate 2 + 3:



Computing

- What do computers do ?
- How do they compute ?
  1. Data movement: read / store
  2. logical : Compare / And / Or
  3. add / subtract / Multiply / Divide / Increment / Decrement
  4. control flow: Jump to another instruction
- Example-1: Calculate 2 + 3:



- By itself not very impressive.

Computing

Example-2:

1. Read marks of next student;
2. Add score;
3. Compute Avg;
4. Store result;

Computing

Example-2:

1. Read marks of next student;
2. Add score;
3. Compute Avg;
4. Store result;
5. Repeat steps for every student;
6. Print the final grade list.

Computing

Example-2:

1. Read marks of next student;
2. Add score;
3. Compute Avg;
4. Store result;
5. Repeat steps for every student;
6. Print the final grade list.

- the computer doesn't learn any new operation for each of the instructions

Computing

Example-2:

1. Read marks of next student;
2. Add score;
3. Compute Avg;
4. Store result;
5. Repeat steps for every student;
6. Print the final grade list.

- the computer doesn't learn any new operation for each of the instructions
- each operation is broken down to basic operation blocks, performed in some sequence

What is programming?

- Programming is the process of organising basic operations into simple applications which combine to solve a much larger problem.

What is programming?

- Programming is the process of organising basic operations into simple applications which combine to solve a much larger problem.
- can compute anything using 6 primitives (Turing)
- programming language provides a set of primitive operations
- create ”expressions” which are complex but legal combination of primitives
- write down a sequence of declaration / evaluation usually to meet some end goal.

What is programming?

- Programming is the process of organising basic operations into simple applications which combine to solve a much larger problem.
- can compute anything using 6 primitives (Turing)
- programming language provides a set of primitive operations
- create ”expressions” which are complex but legal combination of primitives
- write down a sequence of declaration / evaluation usually to meet some end goal.
- Complex software is built by combining numerous of these tiny, simple instructions into a carefully organised sequence.

What is a program?

What is a program?

Program

A program is a sequence of definitions and commands

- definitions **evaluated**
- commands **executed**

What is a program?

Program

A program is a sequence of definitions and commands

- definitions **evaluated**
- commands **executed**

Scripts/Shells

- can be typed directly in a shell
- typically stored in a file

Install and use Python through IDLE (Ubuntu)

- `python3 --version`

Install and use Python through IDLE (Ubuntu)

- `python3 --version`
- `sudo apt-get update`

Install and use Python through IDLE (Ubuntu)

- `python3 --version`
- `sudo apt-get update`
- `sudo apt-get install idle-python3.12 python3-pip`

Install and use Python through IDLE (Ubuntu)

- `python3 --version`
- `sudo apt-get update`
- `sudo apt-get install idle-python3.12 python3-pip`
- `idle-python3.12` (start idle)

Where things may go wrong

Syntactic errors:

- common
- easily caught by compilers
- English:

  “sat cat mat” $\rightarrow$ not syntactically valid

  “cat sat on the mat” $\rightarrow$ syntactically valid
- Programming Language:

  “hi” 5 $\rightarrow$ not syntactically valid

  3.2*5 $\rightarrow$ syntactically valid

Where things may go wrong

Static semantic errors:

- syntactically valid strings
- no semantic errors
  1. program crashes, stops running
  2. program runs forever
  3. program gives an answer but different than expected

Content

Objects

Branching and Iterations

OBJECTS

- programs manipulate data objects

OBJECTS

- programs manipulate data objects
- objects have a type
  - Saurav is a human so he can breathe, sleep, etc.
  - An uncle is a relative, so he can give career advice, discuss politics, ask about your grades.
  - Mango is a fruit, so it can be ...

OBJECTS

- programs manipulate data objects
- objects have a type
  - Saurav is a human so he can breathe, sleep, etc.
  - An uncle is a relative, so he can give career advice, discuss politics, ask about your grades.
  - Mango is a fruit, so it can be ...
- Objects are:
  1. Scalar (cannot be subdivided)
  2. non-scalar (have internal structure that can be accessed)

OBJECTS

- programs manipulate data objects
- objects have a type
  - Saurav is a human so he can breathe, sleep, etc.
  - An uncle is a relative, so he can give career advice, discuss politics, ask about your grades.
  - Mango is a fruit, so it can be ...
- Objects are:
  1. Scalar (cannot be subdivided)
  2. non-scalar (have internal structure that can be accessed)

Scalar objects

- `int` - represents integers, e.g. 7

Scalar objects

- `int` - represents integers, e.g. 7
- `float` - represents real numbers. e.g. 3.14159265

Scalar objects

- `int` - represents integers, e.g. 7
- `float` - represents real numbers. e.g. 3.14159265
- `bool` - represents real Boolean values `True` or `False`

Scalar objects

- `int` - represents integers, e.g. 7
- `float` - represents real numbers. e.g. 3.14159265
- `bool` - represents real Boolean values `True` or `False`
- `NoneType` - special and has one value, `None`

Scalar objects

- `int` - represents integers, e.g. 7
- `float` - represents real numbers. e.g. 3.14159265
- `bool` - represents real Boolean values `True` or `False`
- `NoneType` - special and has one value, `None`
- use `type (..)` to see the type of an object

Scalar objects

Type conversion (cast)

- can convert object of one type to another
- `float (3)` converts integer `3` to float `3.0`
- `int (3.9)` truncates float `3.9` to integer `3`

Scalar objects

Type conversion (cast)

- can convert object of one type to another
- `float (3)` converts integer `3` to float `3.0`
- `int (3.9)` truncates float `3.9` to integer `3`

Print to console

- `3 + 2`

Scalar objects

Type conversion (cast)

- can convert object of one type to another
- `float (3)` converts integer `3` to float `3.0`
- `int (3.9)` truncates float `3.9` to integer `3`

Print to console

- `3 + 2`
- `print (3 + 2)`

Variables

Binding Variables and values

- `interest = 1000.0 * 9.8 * 6 / 100.0`

Variables

Binding Variables and values

- `interest = 1000.0 * 9.8 * 6 / 100.0`
- `roi = 9.8`
- `prin = 1000.0`
- `time = 6`

Variables

Binding Variables and values

- `interest = 1000.0 * 9.8 * 6 / 100.0`
- `roi = 9.8`
- `prin = 1000.0`
- `time = 6`
- `interest = prin * roi * time / 100.0`

Variables

Binding Variables and values

- `interest = 1000.0 * 9.8 * 6 / 100.0`
- `roi = 9.8`
- `prin = 1000.0`
- `time = 6`
- `interest = prin * roi * time / 100.0`
- can modify interest by modifying any of the basic components.

Math Functions and Number Methods

1. `round()`, for rounding numbers to some number of decimal places
   - ```
     >>> round(2.3)
     >>> round(3.14159, 3)
     >>> round(2.65, 1.4)
     ```
     ```

Math Functions and Number Methods

1. `round()`, for rounding numbers to some number of decimal places
   - `>>> round(2.3)`
     `>>> round(3.14159, 3)`
     `>>> round(2.65, 1.4)`
2. `abs()`, for getting the absolute value of a number
   - `>>> abs(3)`
     `>>> abs(-5.0)`
3. `pow()`, for raising a number to some power
   - `>>> pow(2, 3)`
     `>>> pow(2, -2)`
     `>>> pow(2, 3, 2)`

Strings

- Many programmers deal with text on a daily basis
- example
  - web developers work with text that gets input from web forms
  - Data scientists process text to extract data and perform things like sentiment analysis, to classify opinions in a body of text.

Strings

- Many programmers deal with text on a daily basis
- example
  - web developers work with text that gets input from web forms
  - Data scientists process text to extract data and perform things like sentiment analysis, to classify opinions in a body of text.
- Collections of text in Python are called strings

Strings

- letters, special characters, spaces, digits

Strings

- letters, special characters, spaces, digits
- enclose in

  `hi = 'hello there'` or

  `hi = "hello there"` or

Strings

- letters, special characters, spaces, digits
- enclose in

```
hi = 'hello there' or
hi = "hello there" or
```

- concatenate strings

```
name = "saurav"
greet = hi + name
greeting = hi + " " + name
```

- String Indexing

```
>>> flavor = "apple pie"
>>> flavor[1]
'p'
```

- In Python—and most other programming languages—counting always starts at zero.

Strings

- To get the character at the beginning of a string, you need to access the character at position 0:

```
>>> flavor[0]
'p'
```

Strings

- To get the character at the beginning of a string, you need to access the character at position 0:

```
>>> flavor[0]
'p'
```

- String Slicing
  - Suppose you need the string containing just the first three letters of the string "apple pie".

```
>>> first_three_letters = flavor[0] +
flavor[1] + flavor[2]
>>> first_three_letters
'app'
```

Strings

- To get the character at the beginning of a string, you need to access the character at position 0:

```
>>> flavor[0]
'p'
```

- String Slicing
  - Suppose you need the string containing just the first three letters of the string ”apple pie”.

```
>>> first_three_letters = flavor[0] +
flavor[1] + flavor[2]
>>> first_three_letters
'app'
```

  - a portion of a string, called a substring

Strings

- To get the character at the beginning of a string, you need to access the character at position 0:

```
>>> flavor[0]
'p'
```

- String Slicing
  - Suppose you need the string containing just the first three letters of the string "apple pie".

```
>>> first_three_letters = flavor[0] +
flavor[1] + flavor[2]
>>> first_three_letters
'app'
```

  - a portion of a string, called a substring

```
>>> flavor = "apple pie"
>>> flavor[0:3]
```

Strings

- To get the character at the beginning of a string, you need to access the character at position 0:

```
>>> flavor[0]
'p'
```

- String Slicing
  - Suppose you need the string containing just the first three letters of the string "apple pie".

```
>>> first_three_letters = flavor[0] +
flavor[1] + flavor[2]
>>> first_three_letters
'app'
```

  - a portion of a string, called a substring

```
>>> flavor = "apple pie"
>>> flavor[0:3]
'app'
```

Strings

Strings Are Immutable

- we can't change them once we've created them

Strings

Strings Are Immutable

- we can't change them once we've created them

```
>>> word = "goal"
>>> word[0] = "f"
```

Strings

Strings Are Immutable

- we can't change them once we've created them

```
>>> word = "goal"
>>> word[0] = "f"
```

- If we want to alter a string, we must create an entirely new string

Converting Strings to Numbers

- `int()` stands for integer and converts objects into whole numbers,
- `float()` stands for floating-point number and converts objects into numbers with decimal points.

Strings

Converting Numbers to Strings

- Sometimes you need to convert a number to a string.
- ```
  >>> num_pancakes = 10
  >>> "I am going to eat " + num_pancakes
  + " pancakes."
  ```
  ```

Strings

Converting Numbers to Strings

- Sometimes you need to convert a number to a string.
- ```
  >>> num_pancakes = 10
  >>> "I am going to eat " + num_pancakes
  + " pancakes."
  ```
- Since `num_pancakes` is a number, Python can't concatenate it with the string "I'm going to eat".
  ```

Strings

Converting Numbers to Strings

- Sometimes you need to convert a number to a string.
- ```
  >>> num_pancakes = 10
  >>> "I am going to eat " + num_pancakes
  + " pancakes."
  ```
- Since `num_pancakes` is a number, Python can't concatenate it with the string "I'm going to eat".
- To build the string, you need to convert `num_pancakes` to a string using `str()`:

  ```
  >>> num_pancakes = 10
  >>> "I am going to eat " +
  str(num_pancakes) + " pancakes."
  ```
  ```

Input / Output

- output

```
>>> x = 1
>>> print(3)
```

Input / Output

- output

```
>>> x = 1
>>> print(3) >>> print(5*text)
```

Interact With User Input

- input

```
>>> text = input("Type anything...  ")
>>> print (text)
```

Input / Output

- output

```
>>> x = 1
>>> print(3) >>> print(5*text)
```

Interact With User Input

- input

```
>>> text = input("Type anything...  ")
>>> print (text)
```

- Working With Strings and Number:
  user input using the `input()` function, the result is always a string.

Content

Objects

Branching and Iterations

Comments

```
import math

principal = 500000
interest = 7.5
duration = 20

r = interest / 12 / 100
n = duration * 12

if r == 0:
    payment = principal / n
else:
    payment = principal * r * (1 + r) ** n / ((1 + r) ** n - 1)

print("Monthly EMI = ", payment, " Ruppes")
```

Comments

A person can understand the syntax, but they may wonder:

Comments

A person can understand the syntax, but they may wonder:

- Why is the interest divided by 12 and then by 100?
- Where did this complicated formula come from?
- Why is there a special case when r == 0?

The code works, but the intent is not obvious.

Comments

```python
import math

# Loan details
principal = 500000        # Loan amount in rupees
interest = 7.5            # Annual interest rate (in percent)
duration = 20             # Loan repayment period (in years)

# Convert the annual interest rate into a monthly decimal rate,
# as the EMI formula works with monthly interest.
r = interest / 12 / 100

# Total number of monthly installments.
n = duration * 12

# If there is no interest, each installment is simply an equal
# share of the principal.
if r == 0:
    payment = principal / n
else:
    # Compute the Equated Monthly Installment (EMI) using the
    # standard loan amortization formula used by banks.
    payment = principal * r * (1 + r) ** n / ((1 + r) ** n - 1)

# Display the monthly payment amount.
print("Monthly EMI =", payment, "Rupees")
```

Comments

What the comments add

- Why the interest rate is converted.
- What n represents.
- Why there is a special case for zero interest.
- Where the formula comes from (the standard EMI formula).

Comments

What the comments add

- Why the interest rate is converted.
- What n represents.
- Why there is a special case for zero interest.
- Where the formula comes from (the standard EMI formula).
- What each input variable means.

Comments

```
# Print "Hello, world"
print("Hello, world")
```

Comments

```
# Print "Hello, world"
print("Hello, world")
```

- No comment is needed in this example.

Comments

```
# Print "Hello, world"
print("Hello, world")
```

- No comment is needed in this example.
- best used to clarify code that may not be easy to understand
- to explain why something is done a certain way

Comments

```
# Print "Hello, world"
print("Hello, world")
```

- No comment is needed in this example.
- best used to clarify code that may not be easy to understand
- to explain why something is done a certain way
- comments should be used sparingly

Comments

```
# Print "Hello, world"
print("Hello, world")
```

- No comment is needed in this example.
- best used to clarify code that may not be easy to understand
- to explain why something is done a certain way
- comments should be used sparingly
- Use comments only when they add value

Functions

- Functions are the building blocks of almost every Python program.

Functions

- Functions are the building blocks of almost every Python program.
- we've already seen how to use several functions, including `print()`, `len()`, and `round()`.

Functions

- Functions are the building blocks of almost every Python program.
- we've already seen how to use several functions, including `print()`, `len()`, and `round()`.
- These are all built-in functions because they come built into the Python language itself.

Functions

- Functions are the building blocks of almost every Python program.
- we've already seen how to use several functions, including `print()`, `len()`, and `round()`.
- These are all built-in functions because they come built into the Python language itself.
- You can also create user-defined functions that perform specific tasks.

Functions

- Functions are the building blocks of almost every Python program.
- we've already seen how to use several functions, including `print()`, `len()`, and `round()`.
- These are all built-in functions because they come built into the Python language itself.
- You can also create user-defined functions that perform specific tasks.
- Functions break code into smaller chunks,

Functions

- Functions are the building blocks of almost every Python program.
- we've already seen how to use several functions, including `print()`, `len()`, and `round()`.
- These are all built-in functions because they come built into the Python language itself.
- You can also create user-defined functions that perform specific tasks.
- Functions break code into smaller chunks, and are great for tasks that a program uses repeatedly

Functions Are Values

- One of the most important properties of a function in Python is that functions are values and can be assigned to a variable.

Functions Are Values

- One of the most important properties of a function in Python is that functions are values and can be assigned to a variable.
- Like any other variable, you can assign any value you want to len:

```
>>> len= "I'm not the len you're
looking for."
>>> len
```

Functions Are Values

- One of the most important properties of a function in Python is that functions are values and can be assigned to a variable.
- Like any other variable, you can assign any value you want to len:

```
>>> len= "I'm not the len you're
looking for."
>>> len
```

- Now `len` has a string value

Functions Are Values

- One of the most important properties of a function in Python is that functions are values and can be assigned to a variable.
- Like any other variable, you can assign any value you want to len:

```
>>> len= "I'm not the len you're
looking for."
>>> len
```

- Now `len` has a string value
- The variable name `len` is a keyword in Python, and even though you can change it's value, it's usually a bad idea to do so.

Functions Are Values

- Changing the value of `len` can make your code confusing

Functions Are Values

- Changing the value of `len` can make your code confusing
- because it's easy to mistake the new `len` for the built-in function.

Functions Are Values

- Changing the value of `len` can make your code confusing
- because it's easy to mistake the new `len` for the built-in function.
- If you typed in the previous code examples, you no longer have access to the built-in `len` function in IDLE.

Functions Are Values

- Changing the value of `len` can make your code confusing
- because it's easy to mistake the new `len` for the built-in function.
- If you typed in the previous code examples, you no longer have access to the built-in `len` function in IDLE.
- You can get it back with the following code:

```
>>> del len
```

Functions Are Values

- Changing the value of `len` can make your code confusing
- because it's easy to mistake the new `len` for the built-in function.
- If you typed in the previous code examples, you no longer have access to the built-in `len` function in IDLE.
- You can get it back with the following code:

```
>>> del len
```

- The del keyword is used to un-assign a variable from a value.
- `del` stands for delete, but it doesn't delete the value. Instead, it detaches the name from the value and deletes the name.

How Python Executes Functions

- Typing just the name doesn't execute the function.

How Python Executes Functions

- Typing just the name doesn't execute the function.
- must call the function to tell Python to actually execute it.

```
>>> len()
```

How Python Executes Functions

- Typing just the name doesn't execute the function.
- must call the function to tell Python to actually execute it.

```
>>> len()
```

- Python raises a TypeError when `len()` is called because `len()` expects an argument.

How Python Executes Functions

- Typing just the name doesn't execute the function.
- must call the function to tell Python to actually execute it.

```
>>> len()
```

- Python raises a TypeError when `len()` is called because `len()` expects an argument.
- An argument is a value that gets passed to the function as input.

How Python Executes Functions

- Typing just the name doesn't execute the function.
- must call the function to tell Python to actually execute it.

```
>>> len()
```

- Python raises a TypeError when `len()` is called because `len()` expects an argument.
- An argument is a value that gets passed to the function as input.
- Some functions can be called with no arguments, and some can take as many arguments as you like.

How Python Executes Functions

- Typing just the name doesn't execute the function.
- must call the function to tell Python to actually execute it.

```
>>> len()
```

- Python raises a TypeError when `len()` is called because `len()` expects an argument.
- An argument is a value that gets passed to the function as input.
- Some functions can be called with no arguments, and some can take as many arguments as you like.
- When a function is done executing, it returns a value as output.

How Python Executes Functions

The process for executing a function can be summarized in three steps:

1. The function is called, and any arguments are passed to the function as input.

How Python Executes Functions

The process for executing a function can be summarized in three steps:

1. The function is called, and any arguments are passed to the function as input.
2. The function executes, and some action is performed with the arguments.

How Python Executes Functions

The process for executing a function can be summarized in three steps:

1. The function is called, and any arguments are passed to the function as input.
2. The function executes, and some action is performed with the arguments.
3. The function returns, and the original function call is replaced with the return value.

```
>>> num_letters = len("four")
```

- When a function changes or affects something external to the function itself, it is said to have a side effect.

  ```
  >>> return_value= print("What do I return?")
  ```

User define Functions

- Repetitive code can be a nightmare to maintain.

User define Functions

- Repetitive code can be a nightmare to maintain.
- If you find a mistake in some code that's been copied and pasted all over the place, you'll end up having to apply the fix everywhere the code was copied.

The Anatomy of a Function

Every function has two parts:

1. The **function signature** defines the name of the function and any inputs it expects.

User define Functions

- Repetitive code can be a nightmare to maintain.
- If you find a mistake in some code that's been copied and pasted all over the place, you'll end up having to apply the fix everywhere the code was copied.

The Anatomy of a Function

Every function has two parts:

1. The **function signature** defines the name of the function and any inputs it expects.
2. The **function body** contains the code that runs every time the function is used.

Multiplication function

- write a function that takes two numbers as input and returns their product.

```
def multiply(x, y):   # Function signature
  # Function body
  product = x * y
  return product
```

Multiplication function

- write a function that takes two numbers as input and returns their product.

```
def multiply(x, y):  # Function signature
  # Function body
  product = x * y
  return product
```

- The first line of code in a function is called the function signature.
- It always starts with the `def` keyword, which is short for “define.”

Function Signature

```
def multiply(x, y):
```

The function signature has four parts:

- The `def` keyword

Function Signature

```
def multiply(x, y):
```

The function signature has four parts:

- The `def` keyword
- The function name, `multiply`

Function Signature

```
def multiply(x, y):
```

The function signature has four parts:

- The `def` keyword
- The function name, `multiply`
- The parameter list, `(x, y)`

Function Signature

```
def multiply(x, y):
```

The function signature has four parts:

- The `def` keyword
- The function name, `multiply`
- The parameter list, `(x, y)`
- A colon `(:)` at the end of the line

CONTROL FLOW - BRANCHING

Indentation in Python

What is indentation?

- Indentation means adding spaces (or a tab) at the beginning of a line.

Indentation in Python

What is indentation?

- Indentation means adding spaces (or a tab) at the beginning of a line.
- In most programming languages, indentation is used mainly to improve readability.

Indentation in Python

What is indentation?

- Indentation means adding spaces (or a tab) at the beginning of a line.
- In most programming languages, indentation is used mainly to improve readability.
- In Python, indentation is part of the language syntax.

Indentation in Python

What is indentation?

- Indentation means adding spaces (or a tab) at the beginning of a line.
- In most programming languages, indentation is used mainly to improve readability.
- In Python, indentation is part of the language syntax.
- It tells Python which statements belong together as a block.

```
age = 18

if age >= 18:
    print("Eligible to vote")

print("Program finished")
```

Indentation in Python

What is indentation?

- Indentation means adding spaces (or a tab) at the beginning of a line.
- In most programming languages, indentation is used mainly to improve readability.
- In Python, indentation is part of the language syntax.
- It tells Python which statements belong together as a block.

```
age = 18

if age >= 18:
    print("Eligible to vote")

print("Program finished")
```

- In Python, indentation is mandatory, not just a matter of style.

Indentation in Python

Incorrect Indentation

```
marks = 45

if marks >= 50:
     print("Marks = ", marks)
print("Pass")
```